ProfileBook

Capstone Project Report

Angular + ASP.NET Core + SQL Server

Sprint 2

Submitted by: Shubham Agarwal
GitHub: https://github.com/ShubhamAgarwal1202-del/project
Date: 26 March 2026

1. Project Overview

ProfileBook is a full-stack social media web application built using Angular (frontend), ASP.NET Core Web API (backend), and SQL Server (database). It allows users to register, create posts, like/comment on posts, send messages, and report other users. Admins can manage users, approve posts, view reports, and create groups.

Tech Stack

- Frontend: Angular 21 (Standalone Components)
- Backend: ASP.NET Core Web API (.NET 8)
- Database: SQL Server (Entity Framework Core ORM)
- Authentication: JWT Bearer Tokens
- API Documentation: Swagger UI
- Unit Testing: Vitest (Angular built-in)
- Version Control: Git / GitHub

2. Use Case Document

2.1 Actors

Actor	Description
User	Registered member who can create posts, like, comment, message, and report other users
Admin	Platform administrator who manages users, approves posts, views reports, and manages groups
Guest	Unregistered visitor who can only view the login and register pages

2.2 User Use Cases

Use Case	Actor	Description	Precondition
Register	Guest	Create a new account with username and password	Not logged in
Login	User/Admin	Authenticate using username and password to receive JWT token	Registered account exists
View Feed	User	View all approved posts on the main feed	Logged in
Create Post	User	Submit a new post for admin approval	Logged in
Like Post	User	Like or unlike an approved post	Logged in, post exists
Comment on Post	User	Add a comment to an approved post	Logged in, post exists
Send Message	User	Send a private message to another user	Logged in, recipient exists
View Messages	User	View conversation history with another user	Logged in
Search Users	User	Search for other users by username	Logged in

Use Case	Actor	Description	Precondition
Report User	User	Report another user with a reason	Logged in
Logout	User	Clear session and return to login page	Logged in

2.3 Admin Use Cases

Use Case	Actor	Description	Precondition
Admin Login	Admin	Login with admin credentials to access admin dashboard	Admin account exists
View All Users	Admin	See a list of all registered users	Logged in as Admin
Create User	Admin	Create a new user account directly	Logged in as Admin
Update User	Admin	Edit username or role of an existing user	Logged in as Admin
Delete User	Admin	Remove a user from the platform	Logged in as Admin
Approve Post	Admin	Approve a pending post so it appears on the feed	Logged in as Admin
Reject Post	Admin	Reject a pending post	Logged in as Admin
View Reports	Admin	See all user reports with reporter and reason	Logged in as Admin
Dismiss Report	Admin	Delete/dismiss a resolved user report	Logged in as Admin
Create Group	Admin	Create a new user group with members	Logged in as Admin
Delete Group	Admin	Remove an existing group	Logged in as Admin

3. Database Schema

The database ProfileBookDB contains 7 tables created using Entity Framework Core migrations. All relationships follow a normalized relational design.

3.1 Users Table

Column	Data Type	Constraints	Description
UserId	int	PRIMARY KEY, IDENTITY	Unique user identifier
Username	nvarchar(max)	NOT NULL	User's login name
Password	nvarchar(max)	NOT NULL	User's password
Role	nvarchar(max)	NOT NULL, DEFAULT 'User'	User role: 'User' or 'Admin'
ProfileImage	nvarchar(max)	NULLABLE	Path to profile image file

3.2 Posts Table

Column	Data Type	Constraints	Description
PostId	int	PRIMARY KEY, IDENTITY	Unique post identifier
UserId	int	FOREIGN KEY → Users	Author of the post
Content	nvarchar(max)	NOT NULL	Text content of the post
PostImage	nvarchar(max)	NULLABLE	Optional image path
Status	nvarchar(max)	DEFAULT 'Pending'	Pending / Approved / Rejected

3.3 Messages Table

Column	Data Type	Constraints	Description
MessageId	int	PRIMARY KEY, IDENTITY	Unique message identifier
SenderId	int	FOREIGN KEY → Users	User who sent the message
ReceiverId	int	FOREIGN KEY → Users	User who received the message
MessageContent	nvarchar(max)	NOT NULL	Text of the message
Timestamp	datetime2	NOT NULL	When message was sent (UTC)

3.4 Reports Table

Column	Data Type	Constraints	Description
ReportId	int	PRIMARY KEY, IDENTITY	Unique report identifier
ReportedUserId	int	FOREIGN KEY → Users	User being reported
ReportingUserId	int	FOREIGN KEY → Users	User who filed the report
Reason	nvarchar(max)	NOT NULL	Reason for report
Timestamp	datetime2	NOT NULL	When report was filed (UTC)

3.5 Groups Table

Column	Data Type	Constraints	Description
GroupId	int	PRIMARY KEY, IDENTITY	Unique group identifier
GroupName	nvarchar(max)	NOT NULL	Name of the group
GroupMembers	nvarchar(max)	NOT NULL	Comma-separated member names

3.6 Likes Table

Column	Data Type	Constraints	Description
LikeId	int	PRIMARY KEY, IDENTITY	Unique like identifier
PostId	int	FOREIGN KEY → Posts	Post that was liked
UserId	int	FOREIGN KEY → Users	User who liked the post

3.7 Comments Table

Column	Data Type	Constraints	Description
CommentId	int	PRIMARY KEY, IDENTITY	Unique comment identifier
PostId	int	FOREIGN KEY → Posts	Post being commented on
UserId	int	FOREIGN KEY → Users	User who wrote the comment
Content	nvarchar(max)	NOT NULL	Text of the comment
Timestamp	datetime2	NOT NULL	When comment was posted (UTC)

4. Database Relationships

Relationship	Type	Description
Users → Posts	One-to-Many	One user can create many posts
Users → Messages (Sender)	One-to-Many	One user can send many messages
Users → Messages (Receiver)	One-to-Many	One user can receive many messages
Users → Reports (Reported)	One-to-Many	One user can be reported multiple times
Users → Reports (Reporting)	One-to-Many	One user can file multiple reports
Users → Likes	One-to-Many	One user can like many posts
Users → Comments	One-to-Many	One user can comment on many posts
Posts → Likes	One-to-Many	One post can have many likes
Posts → Comments	One-to-Many	One post can have many comments

5. ER Diagram

The following shows the entity relationships in the ProfileBook database:

```
USERS (UserId PK, Username, Password, Role, ProfileImage)
  |-- 1:N --> POSTS (PostId PK, UserId FK, Content, PostImage, Status)
  |             |-- 1:N --> LIKES (LikeId PK, PostId FK, UserId FK)
  |             |-- 1:N --> COMMENTS (CommentId PK, PostId FK, UserId FK,
Content, Timestamp)
  |-- 1:N --> MESSAGES (MessageId PK, SenderId FK, ReceiverId FK,
MessageContent, Timestamp)
  |-- 1:N --> REPORTS (ReportId PK, ReportedUserId FK, ReportingUserId FK,
Reason, Timestamp)
  |-- 1:N --> GROUPS (GroupId PK, GroupName, GroupMembers)
```

6. API Endpoints

All endpoints are documented and testable via Swagger UI at http://localhost:5209/swagger

6.1 Authentication API

Method	Endpoint	Description
POST	/api/Auth/register	Register a new user (username, password, role)
POST	/api/Auth/login	Login and receive JWT token

6.2 User Management API

Method	Endpoint	Description
GET	/api/User/all	Get all users (Admin)
GET	/api/User/search?username=x	Search users by username
POST	/api/User/create	Admin creates a new user
PUT	/api/User/update/{id}	Admin updates user details
DELETE	/api/User/delete/{id}	Admin deletes a user

6.3 Post Management API

Method	Endpoint	Description
POST	/api/Post/create	User submits a new post for approval
GET	/api/Post/approved	Get all approved posts (feed)
GET	/api/Post/pending	Get all pending posts (Admin)
PUT	/api/Post/approve/{id}	Admin approves a post
PUT	/api/Post/reject/{id}	Admin rejects a post
DELETE	/api/Post/delete/{id}	Delete a post

6.4 Likes & Comments API

Method	Endpoint	Description
POST	/api/LikeComment/like	User likes a post
DELETE	/api/LikeComment/unlike	User unlikes a post
GET	/api/LikeComment/likes/{postId}	Get like count for a post
POST	/api/LikeComment/comment	User adds a comment to a post
GET	/api/LikeComment/comments/{postId}	Get all comments for a post (with usernames)
DELETE	/api/LikeComment/comment/delete/{id}	Delete a comment

6.5 Messaging API

Method	Endpoint	Description
POST	/api/Message/send	Send a message to another user
GET	/api/Message/conversation	Get conversation between two users
GET	/api/Message/inbox/{userId}	Get all messages in user's inbox

6.6 Reports API

Method	Endpoint	Description
POST	/api/Report/create	User reports another user
GET	/api/Report/all	Admin views all reports
DELETE	/api/Report/delete/{id}	Admin dismisses a report

6.7 Groups API

Method	Endpoint	Description
POST	/api/Group/create	Admin creates a new group
GET	/api/Group/all	Get all groups
DELETE	/api/Group/delete/{id}	Admin deletes a group

7. Angular Frontend Components

Component	Route	Description
LoginComponent	/login	User and admin login with JWT authentication
Register	/register	New user registration with client-side validation
Feed	/feed	Post feed with like/unlike count and comment features
CreatePost	/create-post	Submit new post for admin approval

Component	Route	Description
Messages	/messages	Two-way chat messaging between users
SearchUsers	/search	Search users by username and report them
AdminDashboard	/admin	Admin control panel with 4 management sections
ManageUsers	/admin/users	Admin CRUD operations on users
ApprovePosts	/admin/posts	Admin approves or rejects pending posts
ReportedUsers	/admin/reported	Admin views and dismisses user reports with usernames
ManageGroups	/admin/groups	Admin creates and deletes groups
Navbar	shared	Reusable navigation component

7.1 Route Guards

Two route guards protect the application:

- authGuard — protects all user routes (/feed, /create-post, /messages, /search). Redirects unauthenticated users to /login.
- adminGuard — protects all admin routes (/admin, /admin/users, /admin/posts, /admin/reported, /admin/groups). Requires both authentication and Admin role.

7.2 Client-Side Validation

Form	Validation Rules
Login	Username required, Password required, Password min 4 characters
Register	Username required, Username min 3 characters, Password min 4 characters, Passwords must match
Create Post	Content required, Content min 5 characters, Content max 500 characters

8. Session Handling & Security

JWT tokens are stored in localStorage after successful login. The token is attached to all API requests via Authorization: Bearer header in Angular HTTP services. Route guards (authGuard and adminGuard) protect all private routes and redirect unauthenticated users to the login page. Admin routes are additionally protected by role verification.

Item	Implementation
Token Storage	localStorage (token, userId, username, role)
Token Transmission	Authorization: Bearer <token> header on all API calls
Token Expiry	7 days (configured in JWT settings)
User Route Protection	authGuard checks isLoggedIn()
Admin Route Protection	adminGuard checks isLoggedIn() AND role === 'Admin'

Item	Implementation
Logout	localStorage.clear() — removes all session data
Note	Password hashing (BCrypt) planned for Sprint 3

9. Sprint Deliverables

9.1 Sprint 1 Deliverables

Deliverable	Status	Details
Use Case Document	Complete	User and Admin use cases documented in Section 2
Database Schema (ER Diagram)	Complete	7 tables with relationships — Section 3 & 5
Database .sql Script	Complete	DataBase_Schema-ProfileBook.sql included in repository
Static views for Post Creation	Complete	CreatePost component with validation
Static views for Messaging	Complete	Messages component with two-way chat
Static views for User Authentication	Complete	Login and Register components

9.2 Sprint 2 Deliverables

Deliverable	Status	Details
Angular components for User UI	Complete	Feed, CreatePost, Messages, SearchUsers
Angular components for Admin UI	Complete	Dashboard, ManageUsers, ApprovePosts, ReportedUsers, ManageGroups
Web API for User Management	Complete	GET/POST/PUT/DELETE /api/User
Web API for Posting	Complete	Create, Approve, Reject, Feed endpoints
Web API for Messaging	Complete	Send, Conversation, Inbox endpoints
JWT-based Authentication	Complete	Login returns token, all routes protected
Separate Web API for Messaging	Complete	/api/Message — independent messaging controller

9.3 Sprint 3 Deliverables

This sprint focused on enhancing user experience, improving system interactivity, and adding real-time engagement features to make the application closer to a real-world social media platform

Notification System
A complete notification system was implemented to keep users informed about activities related to them.
Features:

- Users receive notifications for:
  - Likes on their posts
  - Comments on their posts
  - Friend requests
- Notifications are stored in the database and fetched via API
- Unread notification count is displayed in the UI (badge system)
- Users can view all notifications on a dedicated page

Backend:

- New Notification model and table added
- NotificationController created with:
  - GET /api/Notification → fetch user notifications
  - POST /api/Notification/mark-read/{id} → mark as read
- Notifications are triggered inside:
  - Like API
  - Comment API
  - Friendship API

Frontend:

- NotificationService added
- Notification page created (/notifications)
- Notification bell icon with unread count added in feed/navbar

---

Friendship System Enhancement
The friendship system was completed and integrated across the application.
Features:

- Send friend request
- Accept / reject requests
- View friends list
- Integration with messaging system (chat only between friends)

Improvements:

- Backend APIs stabilized
- UI updated for better interaction
- Friend-based filtering in messaging

Messaging Improvements
Enhancements were made to the messaging system for better usability.
Features:

- Real-time-like chat experience (auto refresh)
- Friend list integration in messaging panel
- Improved UI for conversations
- Error handling for empty chats

UI/UX Improvements (Facebook-style Design)
The frontend UI was significantly upgraded to improve user experience.
Enhancements:

- Modern feed layout with cards and shadows
- Profile sidebar with user details
- Notification badge UI (red count indicator)
- Improved buttons, spacing, and typography
- Cleaner navigation with icon-based navbar

Security Improvements

- Password hashing implemented using BCrypt
- JWT authentication refined
- Secure route handling enforced
- Backend validation improvements

Backend Stability & Testing

- Fixed API issues (connection errors, 500 errors)
- Improved error handling across controllers
- Validated all endpoints via Swagger
- Ensured smooth integration between frontend and backend

Performance Improvements

- Optimized API calls
- Reduced unnecessary UI re-renders
- Improved state handling in Angular components

10. Unit Testing

Unit tests were written using Angular's built-in Vitest testing framework. Tests cover services, components, and validation logic.

Test File	Tests	What is Tested
auth.spec.ts	5	Token save/retrieve, login/logout, isLoggedIn, API calls
login.spec.ts	4	Component creation, validation errors, button rendering
register.spec.ts	4	Component creation, password match validation
feed.spec.ts	1	Component creation and initialization
create-post.spec.ts	1	Component creation
messages.spec.ts	1	Component creation
search-users.spec.ts	1	Component creation
approve-posts.spec.ts	1	Component creation
reported-users.spec.ts	1	Component creation
manage-users.spec.ts	1	Component creation
manage-groups.spec.ts	1	Component creation
admin-dashboard.spec.ts	1	Component creation

Test File	Tests	What is Tested
Services (post, user, message, report, group)	5	Service creation tests
Total	27 passing	All tests passing